CS4475 Fall 2026
Kevin Pinon
Kpinon2@miners.utep.edu

Lab Report
Submitted on 10/16/26

# PA2: Priority-based Scheduler for xv6

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

- Kernel/pstat.h: added to pstat.h to define struct pstat.
- Kernal/sysproc.c: Added sys_getprocs() to extract user-space buffer arguments and invoke getprocs().
- Kernal/proc.c: Implemented getprocs() process table iteration and state copying under lock protection.
- Kernel/syscall.h & syscall.c: Defined and registered SYS_getprocs.
- User/user.h & usys.pl: Added the user system call prototype and generated the assembly stub.

```
$ ps
pid     state           size    age     priority        ppid    name
1       sleeping        16384           0               0       init
2       sleeping        20480           0               1       sh
5       running         16384           0               2       ps
$
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

- Updated allocproc() in kernel/proc.c so every process initializes with a default priority of 0.
- Updated kfork() so child processes inherit their parent's priority value (np->priority = p->priority).
- Added system call handlers sys_getpriority() and sys_setpriority() in kernel/sysproc.c to retrieve and modify priorities.
- Updated getprocs() in kernel/proc.c to include p->priority when populating the struct pstat array.
- Modified user/ps.c to read pstat.priority and output it in a formatted column.

```
$ pexec 15 ps
pid     state           size    age     priority        ppid    name
1       sleeping        16384           0               0       init
2       sleeping        20480           0               1       sh
6       sleeping        16384           15              2       pexec
7       running         16384           15              6       ps
$
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

- I added "uint readytime" to "struct proc" in kernel/proc.h. "readytime" is initialized to 0 in allocproc(). "readytime" is set to the current system tick count whenever a process transitions into the RUNNABLE state. Inside user/ps.c the age for RUNNABLE processes is calculated dynamically. Non-runnable processes leave the age column blank.

```
$ pexec 5 matmul 5 &; matmul 10 &
$ pexec 10 psTime: 18 ticks

pid     state           size    age     priority        ppid    name
1       sleeping        16384           0               0       init
2       sleeping        20480           0               1       sh
13      sleeping        16384           10              2       pexec
11      runnable        16384   21      0               1       matmul
14      running         16384           10              13      ps
$ Time: 56 ticks
```

Task 4. (15 pts) Implement a priority-based scheduler.

- Kernel/param.h: Added compile-time scheduler policy definitions SCHED_RR, SCHED_PRIO, SCHED_AGED and set "#define SCHEDULER SCHED_PRIO"
- Kernel/proc.c: In scheduler() replaced standard round-robin with a priority search algorithm under preprocessor directives.
  - Pass 1 iterates through proc[] and acqures locks to find the RUNNABLE process with the highest priority value, retaining the lock for the best candidate.
  - Pass 2 context-switches to the highest-priority process with swtch().

```
$ pexec 5 matmull 10 &; pexec 10 ps &
$ pid   state           size    age     priority        ppid    name
1       sleeping        16384           0               0       init
2       sleeping        20480           0               1       sh
29      running         16384           10              28      ps
28      sleeping        16384           10              1       pexec
27      runnable        86016   0       0               1       sh
```

What I learned:

- I learned how an operating system kernel decides which process to execute next instead of just letting processes run sequentially. Implementing the priority scheduler helped me understand how context switching works with swtch() and why locks (p->lock) are

critical during scheduling passes. I also learned how preprocessor directives (#if, #else, #endif) allow kernels to switch between different scheduling policies at compile time.

Difficulties:

- The biggest challenge was managing process locks (p->lock) correctly while scanning through the process table for the highest-priority process. At first, my scheduler either deadlocked or crashed QEMU because I was forgetting to release the lock of the previously selected process when finding a new process with a higher priority.

Overcoming the difficulties:

- I drew out the process loop state on paper to trace when locks were acquired and released. I updated the loop logic to explicitly release the lock on `highest_p` whenever a higher-priority process was found, ensuring that exactly one process lock remained held when exiting the selection pass.

Task 5. (15 pts) Add aging to your priority-based scheduler.

- To fix this I used AI(Gemini) to trace the lock states across the process selection loop. With its help, I identified where locks were being leaked and updated the logic to release `highest_p`'s lock whenever a higher-priority process was found, which made sure only one lock remained held when exiting the pass.

```
$ pexec 5 matmul 100 &
$ pexec 10 ps
pid     state           size    age     priority        ppid    name
1       runnable        16384   30      0               0       init
2       sleeping        20480           20              1       sh
40      runnable        16384   30      5               39      matmul
39      sleeping        16384           5               1       pexec
41      sleeping        16384           10              2       pexec
42      running         16384           10              41      ps
$ ps
pid     state           size    age     priority        ppid    name
1       runnable        16384   87      0               0       init
2       sleeping        20480           20              1       sh
40      runnable        16384   87      5               39      matmul
39      sleeping        16384           5               1       pexec
43      running         16384           20              2       ps
$
```

What I learned:

- I learned how dynamic priority prevent process starvation while maintaining fairness and priority preference in operating system schedulers.

Difficulties:

- age remained stuck at 0 because p->readytime = ticks; was executing inside yield() on every timer interrupt tick, resetting the age counter.

Overcoming the difficulties:

- Removed p->readytime = ticks; from yield() ensuring readytime is strictly set when entering the RUNNABLE state from a non-running state. Although simple enough, it took me a bit to understand why this what happening, so I used AI(Gemini) to help me find the issue by asking as to why my code was returning 0.

Task 6. (5 pts) Summary

This assignment built directly on my initial experience with xv6. Learning how the kernel selects processes, tracks wait times across timer interrupts and handles priorities with aging was another steep learning curve. My main difficulties were understanding how process locks (p->lock) work during scheduler table scans to prevent deadlocks, tracking down why wait timers (readytime) were getting constantly reset, and formulating an aging algorithm along with handling general C/kernel syntax errors. To overcome this and learn in the process, I used AI (Gemini) to help explain kernel lock ownership, trace state transitions, syntax, and debugging. Through this process, I gained practical experience implementing priority-based scheduling policies and dynamic aging algorithms. While I did turn in this assignment late, I did not use that as an excuse to rush through it with AI and took the time I needed to learn what I could.

Extra Credit Task (10 points).

Implementation:

- I implemented getpriority and setpriority as built-in commands directly inside user/sh.c similar to how cd is structured. Because built-in shell commands execute within the parent shell process itself rather than spawning a separate child process via fork(), running setpriority modifies the shell's priority directly in memory. When sh subsequently executes any new program, the child process automatically inherits this updated priority through kfork().

How it improves response times:

```
$ getpriority
0
$ setpriority 20
$ getpriority
20
$ pexec 5 matmul 100 &
$ ps
pid     state           size    age     priority        ppid    name
1       runnable        16384   10      0               0       init
2       sleeping        20480           20              1       sh
46      runnable        16384   9       5               45      matmul
45      sleeping        16384           5               1       pexec
47      running         16384           20              2       ps
$ Time: 375 ticks
```

As shown in the output, process 2 (`sh`) has been set to a priority of `20` using `setpriority`. Because the shell runs at priority `20`, any command executed from it inherits priority `20`. When the background process 46 (`matmul`) is launched at priority 5, it runs in the background while

waiting in the `runnable` state. Because the shell and `ps` operate at priority `20` which is higher than `matmul`'s priority `5`, the kernel schedules `ps` immediately without waiting for `matmul` to finish. This allows the terminal to return instantly without freezing the interface.